



Slide 1

CS 170

Java Programming 1 Working Objects

Duration: 00:01:11

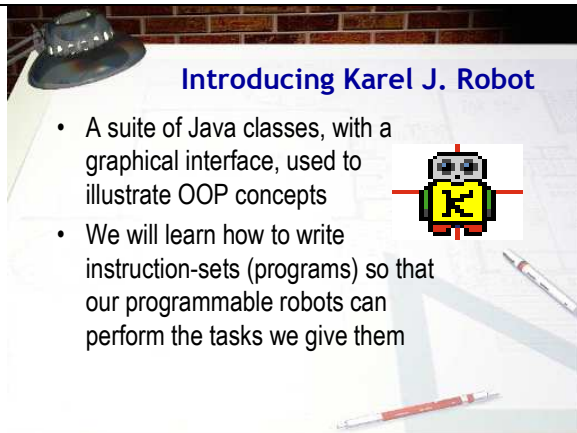
Advance mode: Auto

Notes:

This is the CS 170, Java Programming 1 lecture, "Working Objects". In the 1970s, a Stanford Graduate student named Rich Pattis designed an introductory programming environment, using some of the ideas from the LOGO project at MIT. Instead of teaching children to program, though, Pattis, who is now a professor at Carnegie Mellon, wanted to teach undergraduates the fundamentals of programming by removing all of the non-essentials.

His creation, Karel the Robot, was named after the Czech playwright, Karel Capek who's 1923 play Rossum's Universal Robots, introduced the word robot into the English language. In Karel's world, students learned to program by teaching an on-screen robot to solve simple problems.

Karel the Robot was quite a success. It was used in introductory Computer Science classes all across the country. Later generations of Karel learned Pascal and C++, and, this last year, he's even picked up Python. The version of Karel we'll use, Karel J. Robot, speaks Java.



Slide 2

Introducing Karel J. Robot

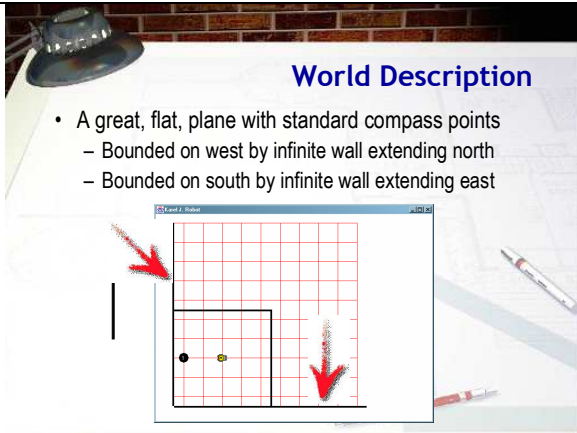
Duration: 00:01:04

Advance mode: Auto

Notes:

Karel J. Robot, is a suite of Java classes, along with a graphical user interface, that we'll use to introduce, illustrate, and give you some practice with, the different object-oriented programming concepts we cover throughout the semester. I've been really pleased with how well it helps to introduce concepts by making those concepts real and concrete, instead of vague and theoretical. Although Karel was originally written by Rich Pattis, the version that we'll use was modified by Professor Stephen Edwards at Virginia Tech, who added the testing features. I took Professor Edwards version and patched it up a little to work here at OCC.

We'll start out by learning how to program Karel to perform a specific task. When we do, we'll only be able to use the commands that Karel understands. Karel programs use the same structure and use the same fundamental syntax that Java programs do; the difference is that Karel's programming language is extremely small; it has only a few commands and rules.



World Description

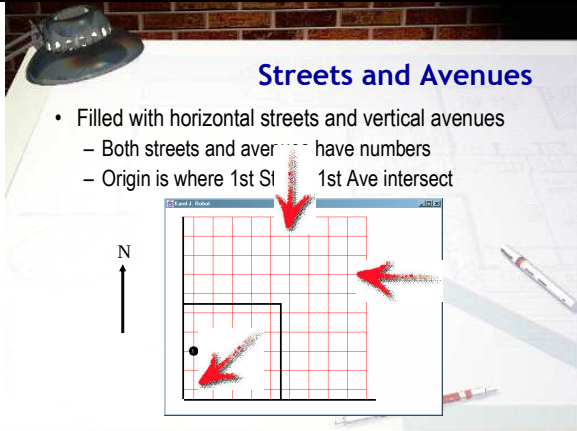
- A great, flat, plane with standard compass points
 - Bounded on west by infinite wall extending north
 - Bounded on south by infinite wall extending east

Slide 3
World Description
 Duration: 00:00:23
 Advance mode: Auto

Notes:

Here's what Karel's world looks like. It is basically a great, flat plane, just like you'd see on a map. Karel's world uses the standard compass points, so north is at the top of the screen.

Notice that on the left (or West) and on the bottom (or South), there's a great wall extending off to infinity.



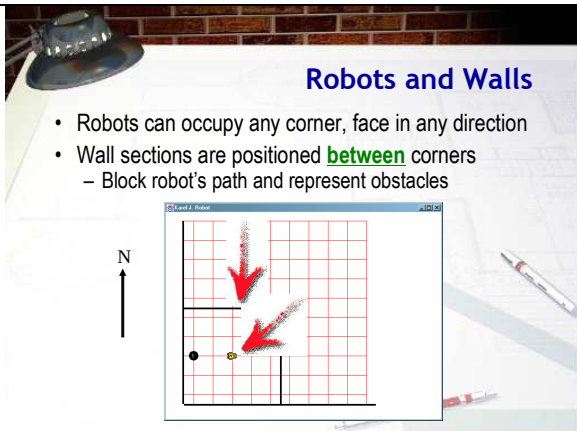
Streets and Avenues

- Filled with horizontal streets and vertical avenues
 - Both streets and avenues have numbers
 - Origin is where 1st St and 1st Ave intersect

Slide 4
Streets and Avenues
 Duration: 00:00:24
 Advance mode: Auto

Notes:

The world is filled with equally-spaced streets and avenues. The streets are horizontal (going from left to right), while the avenues are vertical, going from bottom to top. Each of the streets is numbered, starting at first street and first avenue, in the south-west corner of the world. This intersection is called the "origin".



Robots and Walls

- Robots can occupy any corner, face in any direction
- Wall sections are positioned **between** corners
 - Block robot's path and represent obstacles

Slide 5
Robots and Walls
 Duration: 00:00:50
 Advance mode: Auto

Notes:

Robots can occupy any corner and face in any direction. In this picture, Karel is on the corner of 3rd Street (the horizontal line) and 3rd Avenue (the vertical line), and is facing East. Unless you specify a different location and direction you first create a Robot, it starts at the origin, 1st Street and 1st Avenue, facing North.

Wall sections are the thicker black lines that appear between the corners. Wall sections block the robot's path so it can't go forward. If a robot attempts to pass through a wall, its safety mechanisms automatically shut it off. In Karel's world, walls represent obstacles that you must navigate around when you write your programs.

What are Beepers?

- Small plastic cones that emit a quiet beeping noise
 - Found on corners; don't interfere with robot movement
 - Can be carried, picked up and put down by robots
 - There can be more than one beeper on a corner, and a robot can carry an infinite supply of beepers
 - Used to represent **tasks** for the robots to manipulate

Slide 6

What are Beepers?

Duration: 00:01:09

Advance mode: Auto

Notes:

The last inhabitant of Karel's world is the beeper. According to Pattis, a beeper is a small plastic cone the emits a quiet beeping noise. In our version of Karel, it's simply a black circle with a number in it. If it emits a quiet beeping noise, it's so quiet that I've actually never heard it.

Beepers are found only on corners, but they don't interfere with the robot's progress; robots can walk right over them. Robots can also pick them up, carry them, and put them down. If more than one beeper is located on a corner, then the circle representing the beeper will contain the number. In the picture here, there is only a single beeper on each corner. If there are multiple beepers on a corner, then the robot can only pick up one at a time. The robot can, however, carry an infinite number of beepers.

For our purposes, when we write our programs, we'll ask our robot to carry out a specific task. Most often, those tasks involve manipulating the beepers in the world in some way.

Let's talk a little more about tasks and situations.

Tasks and Situations

- A **task** is something we want the robot to do
 - Escape from an enclosed room that has a door
 - Find a beeper and deposit it on origin
 - Escape from a maze
- A **situation** describes exactly what the world looks like
 - Robot's current position and direction it is facing?
 - Location and length of each wall section?
 - Location of each beeper?
 - How many beepers in the robots beeper bag?

Slide 7

Tasks and Situations

Duration: 00:01:05

Advance mode: Auto

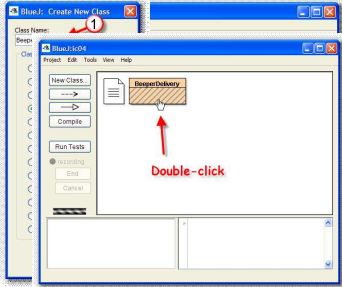
Notes:

When you write programs for Karel, your program will be called a task; the task is what you want the robot to do. Karel may find himself in an enclosed room with only a single door, and you might program him to escape. You may turn him loose to find all of the beepers in the world, collect them, and bring them back to the origin. You may have a random maze, and Karel's task, and the purpose of your program, is to escape.

The world Karel finds himself in when your program starts we call the situation. Karel situations are not part of the program that you'll write. Instead, you'll load the different world situations, (commonly just called "worlds") from a disk file when the task first begins. Our version of Karel actually downloads its situations from the Internet, so you'll have to be connected to run your Karel programs. Situations include things like the robot's position and direction; all of the coordinates of the wall sections; the location of each beeper, and the contents of the robot's beeper bag.

Karel in BlueJ

- Open your current project (if closed) in BlueJ
- Click the **New Class** button
 - **BeeperDelivery**
 - **Robot task** class
 - OK
- **Exercise 1:** pic
- Open in editor



Slide 8
Karel in BlueJ
 Duration: 00:01:13
 Advance mode: Auto

Notes:

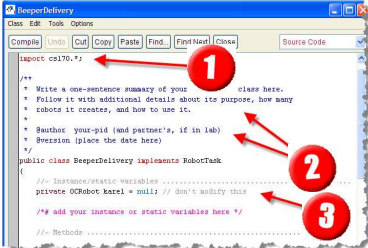
OK, enough theory, let's write your first Karel program using BlueJ. (You must be using the "OCC" version of BlueJ to have access to the Karel classes. If you are using a different version of BlueJ, download the OCC version from the class Web page to your local machine. I've packaged this as a zip file, so all you have to do is to unzip it and then make a shortcut to BlueJ.exe in the BlueJ folder.)

- One you have BlueJ open, reopen this lecture's project, if you've closed it, and then click the New Class button as shown here.
- When the New Class dialog appears, name your class **BeeperDelivery** and select the Robot task template from the list of available templates. When your dialog looks like that in the picture here, click the OK button.

Let's continue now with this week's exercise document. Create a new section for your Karel exercises. In this section we'll restart the numbering. For Exercise 1, snap me a picture and paste it into your document. When you've finished, open the file in the editor.

The Robot Task Template

- Notice the code that is "pre-written" for you
 - cs170 package containing **OCRobot** definition
 - Add your name and date, and a description
- **Exercise 2:** Shoot a pic
- Note the **OCRobot** variable



Slide 9
The Robot Task Template
 Duration: 00:01:15
 Advance mode: Auto

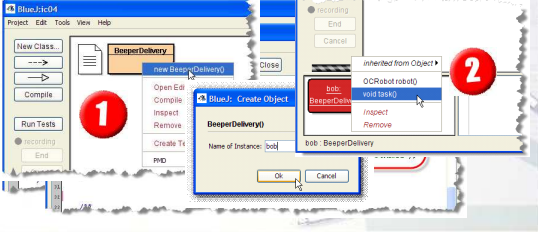
Notes:

The Robot task template contains a lot of code that is pre-written for you:

1. First, the cs170 package is already imported. It contains definitions of an OCRobot class (borrowed from the VPIRobot class which was, itself borrowed from Rich Pettis original Stanford UrRobot class). It also contains some definitions for the World class, mostly developed at Virginia Tech.
2. Below that is a header section, with some notes reminding you to add comments to your code and to add the name and date. For Exercise 2, go ahead and modify that now adding your own information. Our robot will pick up a beeper from the corner of 2nd St. and 1st Avenue, and carry it over to the corner of 4th Street and 2nd Avenue. Then it will go to 1st Street and 3rd Avenue and turn itself off. You can use that as you write your description.
3. Finally, note that the template code already contains an OCRobot variable named karel. In your program, you'll create a robot object, and assign it to this variable.

Starting the World

- Locate the `startFromURL()` method and replace with:
 - "http://csjava.occ.cccd.edu/~gilberts/worlds/ic04.kwld"
- Compile, create `BeeperDelivery` object, and call `task()`



Slide 10
Starting the World
 Duration: 00:01:23
 Advance mode: Auto

Notes:

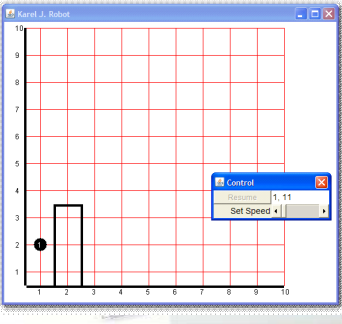
OK, now let's add some code. We'll start by just loading a simple situation. To do that, find the `startFromURL()` method and replace the `String` parameter with "http://csjava.occ.cccd.edu/~gilberts/worlds/ic04.kwld". Don't forget the quotes around the string. The `kwld` extension stands for Karel's world.

To see if you did it correctly, compile your program, create a `BeeperDelivery` object, and call its `task()` method like this:

- Right-click the `BeeperDelivery` class and choose `new BeeperDelivery()`.
- Give your new object a name. In the picture here, I've named my program `bob`, but a name like `mytask` is probably more appropriate.
- Once the object appears in the object bench, right click it and select `task()`.

The BeeperDelivery World

- Exercise 3:**
Shoot a picture



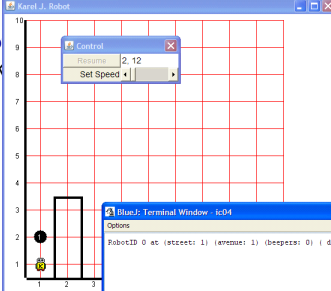
Slide 11
The BeeperDelivery World
 Duration: 00:00:34
 Advance mode: Auto

Notes:

For Exercise 3, shoot a screen-shot of the world. It should look something like this. If you don't see anything, look in your taskbar for a button with the Java icon; sometimes Karel's world appears behind the other windows on your screen. If the World appears, but you don't see the beeper and the little inverted-U section of wall, you've probably spelled the name of the URL incorrectly. If you do this, you'll also see some red error messages in the BlueJ console window as well.

Adding a Robot

- The world has no robots yet. Locate the magenta line that says: `/*# add`
 - Use the `OCRobot`
 - Compile your task
- Exercise 4:**
Shoot a picture of each window that appears



Slide 12
Adding a Robot
 Duration: 00:01:13

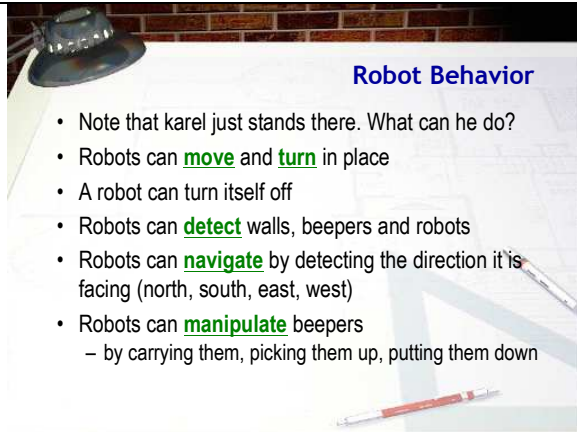
Notes:

The only problem with the program so far is that it doesn't yet have any robots. That's easily remedied. Just locate the magenta line that says "add your code here". (You can delete that line when you add your code.) Then, use the `OCRobot` constructor to initialize the variable named `karel`. If you don't remember how to do this, look back to the first of this week's online lessons. The `OCRobot` constructor doesn't require you to pass any parameters. Make sure when you do this, that you use the existing variable named `karel`, and don't create a new `OCRobot` variable named `karel`.

Compile your task, then create an object and call `task` once again. You should see `karel` sitting on the corner of First Street and First Avenue, facing North. If that's the case, for Exercise 4, shoot a picture of each of the three windows that appears. If

Advance mode: Auto

Karel doesn't appear, ask for some help on the discussion board. You have to complete this part before you can go on; you can't move Karel around the city if you don't actually have an OCRobot object to move.



Robot Behavior

- Note that karel just stands there. What can he do?
- Robots can **move** and **turn** in place
- A robot can turn itself off
- Robots can **detect** walls, beepers and robots
- Robots can **navigate** by detecting the direction it is facing (north, south, east, west)
- Robots can **manipulate** beepers
 - by carrying them, picking them up, putting them down

Slide 13

Robot Behavior

Duration: 00:01:35

Advance mode: Auto

Notes:

So, now that we have a robot, what can we tell him to do. The stock OCRobot as it comes from the factor has a pretty limited range of talents:

- All robots can move and turn in place. They can only move forward, not backwards, and always by one single block. When turning in place, they can only turn left 90 degrees.
- A robot can turn itself off once it's completed its task. Unfortunately, there are no commands for turning it back on again. You'll need to construct a new robot if the one you have gets turned off, even if it turns off by accidentally running into a wall.
- Robots can detect walls, beepers and other robots, but they are really near-sighted. They can only see a wall if it's exactly $\frac{1}{2}$ block directly in front of them, and they can only see other robots or beepers if they are on the same corner. Otherwise, everything is just a blur.
- Robots also have navigation ability, since they can tell which direction they are facing.
- Finally, robots are able to manipulate beepers by picking them up, carrying them, and dropping them off somewhere else. Our OCRobot isn't very fault-tolerant, however. If you ask it to pick up a beeper, and there is no beeper on its corner, it turns itself off. Similarly, if you ask it to drop a beeper, and its beeper-bag is empty, it turns itself off. And, as I mentioned earlier, once an OCRobot gets switched off, it's good only for the scrap heap.



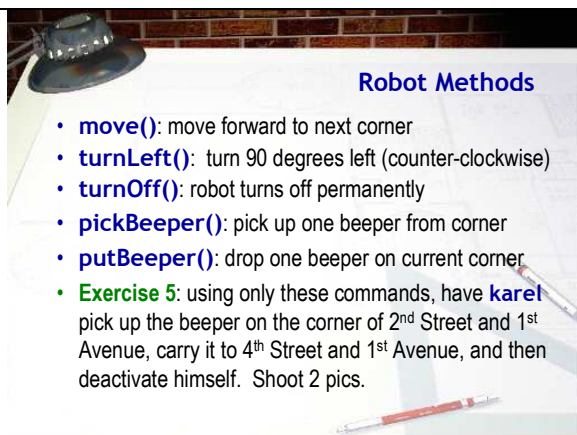
Slide 14
Robot Messages
 Duration: 00:00:59
 Advance mode: Auto

Notes:

Imagine you were a taxi dispatcher talking to all of the taxi drivers over the radio. If you wanted a particular taxi to pick up a fare at the convention center, you'd have to identify the taxi and what you wanted them to do: "Car 35, pick up a party at the convention center."

That's kind of how it works with objects. To ask a particular robot to carry out an action, you have to identify which object you're talking about, and the action you want the object to perform. You do that by starting with the name of the reference; this is called the object receiver. You follow the receiver with a period or dot, and then the message you want to send. Here's an example.

To send a message to an OCRobot object named karel just write this statement. The reference in this case is the variable named karel, while the message is move(). The period separates the two pieces, and, like all Java statements, it ends with a semicolon.



Slide 15
Robot Methods
 Duration: 00:01:34
 Advance mode: Auto

Notes:

Let's take a quick look at the messages that the stock OCRobot responds to.

- move() will go forward one block to the next corner, provided there isn't a wall in the way. If there is a wall, the robot shuts itself off.
- turnLeft() stays on the same corner and turns 90 degrees (counter-clockwise) to the left
- turnOff() disconnects the robot altogether. As mentioned earlier, once a robot turns off, it can't be started back up.
- pickBeeper() picks up one beeper from the current corner. If there is no beeper on the current corner, the robot turns off. If there are multiple beepers, it picks up one.
- putBeeper() takes a beeper out of the robot's beeper bag and deposits it on the current corner. If the robot has no beepers, it shuts down.

OK, now let's put that to work. Using only the commands shown here, add the code needed (right after the line where you create your OCRobot object) to have karel pick up the beeper on the corner of 2nd Street and 1st Avenue, carry it to 4th Street and 1st Avenue, and then turn himself off. You don't need to drop the beeper. We'll finish this program up at our next class meeting. (Make sure you bring it with you when you come to class.) For right now, just shoot me two screen-shots: one of the code you've written and a second screenshot of karel, successfully deactivated on 4th and 1st.

